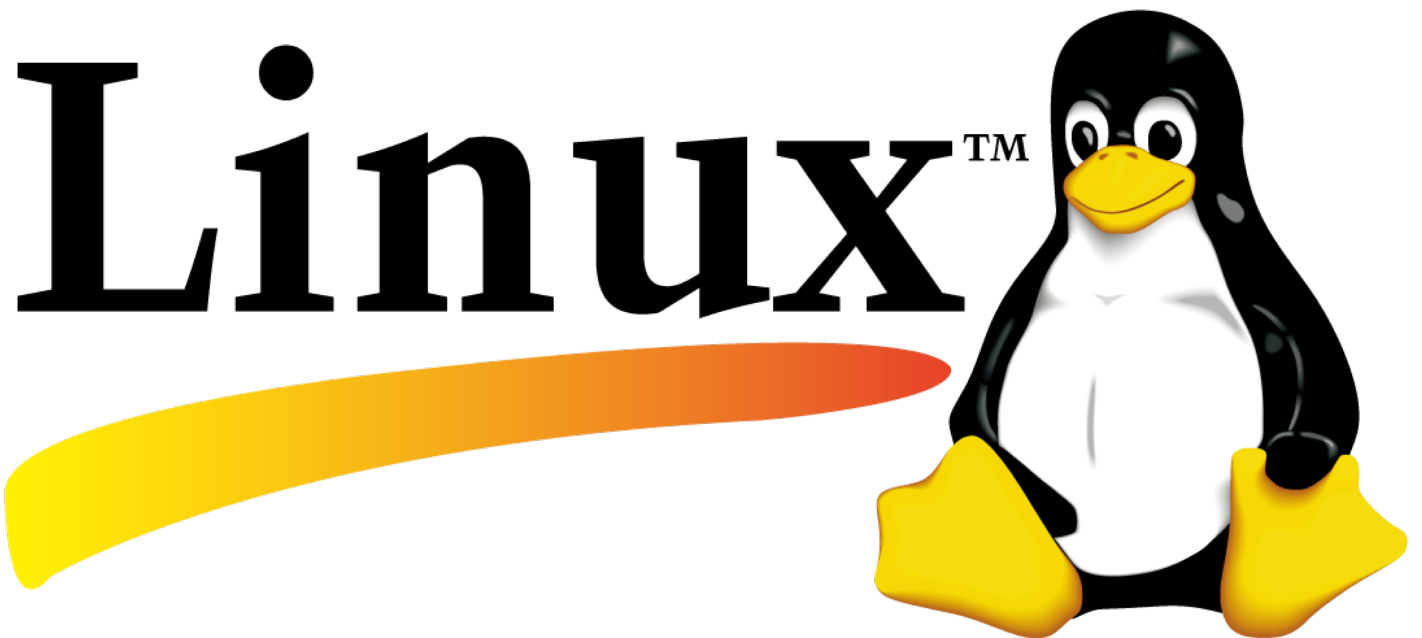




Appunti

Stacchiotti Elia

Indice

1	GitHub	1
1.1	Repository remoti	1
1.2	Creare e gestire repository remoti su GitHub	1
1.3	Creare e gestire repository remoti su GitHub	2
	Fonti	3

Listati

1

GitHub

In questo capitolo viene trattata la piattaforma *GitHub* e, più in particolare, la sua integrazione con il software *Git*, questo binomio è oggi ampiamente diffuso nello sviluppo software, soprattutto nel contesto open source. Come visto nel capitolo ??, *Git* è uno strumento per il versionamento dei file e fornisce meccanismi che si prestano naturalmente allo sviluppo collaborativo, tuttavia, *Git* opera in modo completamente locale e non fornisce direttamente strumenti per la condivisione e la sincronizzazione dei repository tra più utenti. In assenza di una piattaforma remota, la condivisione di un archivio richiederebbe infatti il trasferimento manuale dei dati tra macchine diverse (ad esempio tramite dispositivi fisici), rendendo difficile gestire in modo efficace il lavoro collaborativo.

GitHub nasce proprio per risolvere queste problematiche offrendo un'infrastruttura online che consente di ospitare repository *Git* e di renderli accessibili tramite rete. In questo modo la collaborazione tra più utenti diventa una caratteristica pienamente supportata grazie a strumenti che permettono la sincronizzazione delle modifiche, la gestione dei contributi e il coordinamento dello sviluppo.

1.1. Repository remoti

Con il termine *repository remoto* si indica, in modo generale, un archivio di file che si trova su una macchina diversa rispetto a quella su cui si sta lavorando. L'utilizzo di repository remoti risulta particolarmente utile in diversi scenari: da un lato permette di condividere un progetto tra più macchine o utenti, dall'altro consente di mantenere una copia di sicurezza dell'archivio in una posizione separata.

Nell'integrazione tra *Git* e *GitHub*, il repository remoto rappresenta una versione del repository *Git* ospitata su un server online, ovvero *GitHub*, e il suo scopo principale è quello di permettere la collaborazione tra più utenti. La condivisione, in questo contesto, non si limita alla semplice consultazione o al download dell'archivio, ma comprende anche la possibilità di modificarlo e aggiornare la versione condivisa. In termini operativi, il flusso di lavoro tipico prevede che un utente scarichi una copia del repository, apporti modifiche localmente utilizzando *Git* e successivamente invii tali modifiche al repository remoto, aggiornandone il contenuto. Sebbene questo processo possa apparire semplice, nella pratica emergono problematiche non banali, come la gestione di modifiche simultanee da parte di più utenti. *Git* e *GitHub* forniscono strumenti specifici per affrontare queste situazioni, evitando la perdita di dati e garantendo che le diverse versioni del progetto vengano integrate in modo coerente.

1.2. Creare e gestire repository remoti su GitHub

Per creare una repository remota su *GitHub* è sufficiente creare un account e utilizzare il pulsante "New". Successivamente è possibile specificare il nome della repository e la tipologia, *public* o *private*: nel primo caso la repository è accessibile pubblicamente, mentre nel secondo è visibile e modificabile solo dagli utenti autorizzati.

Una volta completata la creazione, *GitHub* fornisce un URL che identifica univocamente la repository e che rappresenta l'elemento principale per interagire tramite *Git*.

Sebbene GitHub metta a disposizione anche un editor web per creare, modificare ed eliminare file direttamente dal browser, è consigliabile limitarne l'utilizzo, soprattutto nelle fasi iniziali, in quanto può introdurre modifiche non sincronizzate con il repository locale e rendere meno chiaro il flusso di lavoro.

Per iniziare a lavorare sulla repository remota, è possibile clonarla in locale tramite il comando:

```
git clone <URL> <percorso>
```

Questo comando crea una copia completa della repository remota all'interno di una cartella locale, includendo l'intera cronologia dei commit. Nel caso di repository private, potrebbe essere richiesto di effettuare l'autenticazione prima di completare l'operazione.

Dopo il clone, il repository locale è già configurato per comunicare con quello remoto: Git crea automaticamente un riferimento chiamato *origin*, che memorizza l'URL della repository remota.

Per verificare i repository remoti associati, è possibile utilizzare il comando: `git remote -v`

che mostra gli alias configurati e i relativi URL.

Nel caso in cui si voglia collegare manualmente una repository locale a una remota, è possibile utilizzare il comando: `git remote add <alias> <URL>`

Questa operazione è necessaria solo se la repository locale non è stata ottenuta tramite *clone*.

A questo punto, il repository locale e quello remoto risultano collegati, rendendo possibile sincronizzare le modifiche tra i due attraverso i comandi che verranno introdotti nella sezione successiva.

Fonti

- [1] Website Name <OR> I. Surname, I. Surname e I. Surname. *Title of the Website*. 2000. URL: <https://example.com> (visitato il 24/12/2020).